

Name: Abhinav Garg

ID: 2023ucp1706

Ans.1.

```
#include <stdio.h>
```

```
#include <omp.h>
```

```
int computeA() {  
    return 1;  
}
```

```
int computeB() {  
    return 2;  
}
```

```
int main() {  
    int sharedVar;  
    int res = 0;
```

```
    #pragma omp parallel reduction(+:res)  
    {  
        #pragma omp single  
        {  
            sharedVar = 10;  
            printf("Thread %d initialized sharedVar to %d\n", omp_get_thread_num(),  
sharedVar);  
        }  
    }
```

```
    #pragma omp sections  
    {  
        #pragma omp section  
        {  
            int val = computeA() * sharedVar;  
            res += val;  
            printf("Thread %d computed computeA() * sharedVar = %d\n",  
omp_get_thread_num(), val);  
        }  
    }
```

```
    #pragma omp section  
    {  
        int val = computeB() * sharedVar;  
        res += val;  
        printf("Thread %d computed computeB() * sharedVar = %d\n",  
omp_get_thread_num(), val);  
    }  
}
```

```

printf("Result: %d\n", res);

return 0;
}

```

Output:

```

mnitcse@mnitcse-OptiPlex-5040:~/pdc/lab4$ gcc -fopenmp ans1.c -o ans1
mnitcse@mnitcse-OptiPlex-5040:~/pdc/lab4$ ./ans1
Thread 2 initialized sharedVar to 10
Thread 1 computed computeB() * sharedVar: 20
Thread 3 computed computeA() * sharedVar: 10
Result: 30

```

Ans.2.

```

#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
#include <time.h>

#define N 1000000

int privateVar;
#pragma omp threadprivate(privateVar)

int sharedVar = 5;

int computeA() {
    return 1;
}

int computeB() {
    return 2;
}

int main() {
    int i;
    long long serial_dot = 0;
    long long parallel_dot = 0;
    double start, end;
    int *vec1 = (int *)malloc(N * sizeof(int));
    int *vec2 = (int *)malloc(N * sizeof(int));

    srand(time(NULL));

    for (i = 0; i < N; i++) {

```

```

    vec1[i] = rand() % 100 + 1;
    vec2[i] = rand() % 100 + 1;
}

start = omp_get_wtime();

for (i = 0; i < N; i++) {
    serial_dot += vec1[i] * vec2[i];
}

end = omp_get_wtime();
double serial_time = end - start;

start = omp_get_wtime();

#pragma omp parallel
{
    privateVar = omp_get_thread_num() + 1;

    #pragma omp for reduction(+:parallel_dot)
    for (i = 0; i < N; i++) {
        parallel_dot += vec1[i] * vec2[i];
    }
}

end = omp_get_wtime();
double parallel_time = end - start;

int resultA = 0, resultB = 0;
#pragma omp parallel
{
    privateVar = omp_get_thread_num() + 2;
    #pragma omp sections
    {
        #pragma omp section
        {
            resultA = computeA() * sharedVar * privateVar;
        }
        #pragma omp section
        {
            resultB = computeB() * sharedVar * privateVar;
        }
    }
}
#pragma omp single
{
    printf("pragma omp single\n");
}

```

```

}

printf("Serial dot product: %lld\n", serial_dot);
printf("Parallel dot product: %lld\n", parallel_dot);
printf("Section-1 result: %d\n", resultA);
printf("Section-2 result: %d\n", resultB);
printf("Serial time: %f s\n", serial_time);
printf("Parallel time: %f s\n", parallel_time);

free(vec1);
free(vec2);

return 0;
}

```

Output:

```

mnitcse@mnitcse-OptiPlex-5040:~/pdc/lab4$ gcc -fopenmp ans2.c -o ans2
mnitcse@mnitcse-OptiPlex-5040:~/pdc/lab4$ ./ans2
#pragma omp single
Serial dot product: 2549361210
Parallel dot product: 2549361210
Section-1 result: 25
Section-2 result: 40
Serial time: 0.002301 s
Parallel time: 0.000820 s

```

Ans.3.

```

#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
#include <time.h>

#define SIZE 10

int privateVar;
#pragma omp threadprivate(privateVar)

int main() {
    int i, j;
    int mat[SIZE][SIZE];
    int vec[SIZE];
    int result[SIZE] = {0};
    long long totalSum = 0;
    int maxValue = 0;

    srand(time(NULL));

```

```

for (i = 0; i < SIZE; i++) {
    vec[i] = rand() % 100 + 1;
    for (j = 0; j < SIZE; j++) {
        mat[i][j] = rand() % 100 + 1;
    }
}

#pragma omp parallel
{
    privateVar = 0;
    #pragma omp single
    {
        printf("Starting multiplication\n");
    }
    #pragma omp for reduction(+:result[:SIZE])
    for (i = 0; i < SIZE; i++) {
        for (j = 0; j < SIZE; j++) {
            result[i] += mat[i][j] * vec[j] + privateVar;
        }
        privateVar++;
    }
    printf("Thread %d computed %d rows\n", omp_get_thread_num(), privateVar++);
}

#pragma omp parallel sections
{
    #pragma omp section
    {
        for (i = 0; i < SIZE; i++) {
            totalSum += result[i];
        }
    }

    #pragma omp section
    {
        maxValue = result[0];
        for (i = 1; i < SIZE; i++) {
            if (result[i] > maxValue) {
                maxValue = result[i];
            }
        }
    }
}

printf("Sum: %lld\n", totalSum);
printf("Max value: %d\n", maxValue);

```

```
    return 0;  
}
```

Output:

```
mnitcse@mnitcse-OptiPlex-5040:~/pdc/lab4$ gcc -fopenmp ans3.c -o ans3  
mnitcse@mnitcse-OptiPlex-5040:~/pdc/lab4$ ./ans3  
Starting multiplication  
Thread 1 computed 3 rows  
Thread 3 computed 2 rows  
Thread 2 computed 2 rows  
Thread 0 computed 3 rows  
Sum: 104030  
Max value: 50547
```